Vincent Wang

Professor Kambhampaty

CMPSC 132

28 April 2025

Number Guessing Game (Project #3)

My project is based on the third option for the Final Project, which was the Number Guessing Game, a terminal and single player game. The program generates a random integer between 1 and 100 and allows the player to guess it within a limited number of attempts, based on a difficulty level. After each guess, the program provides feedback indicating whether the guess was too high or too low. The game ends when the player guesses correctly or uses up all their attempts. The project also includes two enhancements: a difficulty system (easy, medium, hard) that controls the number of allowed attempts, and a replay feature that lets the player start a new game without restarting the program.

The program is organized into three functions, each responsible for a distinct piece of logic, with a main game loop tying them together.

1. get_valid_number(prompt)

This helper validates the user's input as an integer using a try/except block. If the conversion succeeds, the integer is returned. If the user enters non-numeric text, a ValueError is caught and None is returned instead.

2. get_max_attempts()

This helper prompts the player to select a difficulty level and returns an integer that indicates how many attempts they have. Easy returns 10 attempts, medium returns 6, and hard returns 3. An unrecognized input returns None.

3. number_guessing_game()

This is the main game function. It first asks whether the player wants to play, then enters an outer while loop that drives the replay feature. Inside each iteration, it generates a random number, collects a valid difficulty and first guess (re-prompting as needed), and then enters an inner while loop that continues as long as the guess is wrong and attempts remain. After the loop, an if/else block determines whether to print a congratulations message or reveal the correct answer.

The project does not really use data structures in the sense that anything is being stored inside a stack, list, or linked list. It instead makes use of many loops, and many different data types, such as integers, strings, and the return value of None. It also makes use of having multiple methods instead of one method that runs the entire program.

Several issues arose during the actual coding. The "used up all attempts" message originally printed unconditionally after the inner while loop, even when the player guessed correctly. This was fixed by adding an if/else check on the final value of user_guess after the loop exits. Another issue was the input validation crash Typing non-numeric input caused an unhandled ValueError and crashed the program. I used an int() conversion in a try/except and returning None resolved this. The final issue was an unset max_attempts variable. When an invalid difficulty was entered, max_attempts remained 0 and the game loop never executed. I separated the difficulty selection into its own helper with None-based validation.

There are some possible improvements for this program. One is a high score tracking, which stores the fewest attempts across sessions and will output it to the terminal. Another is a hint system, such as "The number is even”, which can be outputted after a certain number of wrong guesses to make the game more engaging. Finally, allowing the player to set a custom range would allow it to be more personable.